COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structure and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	July 23 2026	**DATE SUBMITTED:**	July 28 2026
NAME:	Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Table 3-1. Output of Initial/Simple Implementation

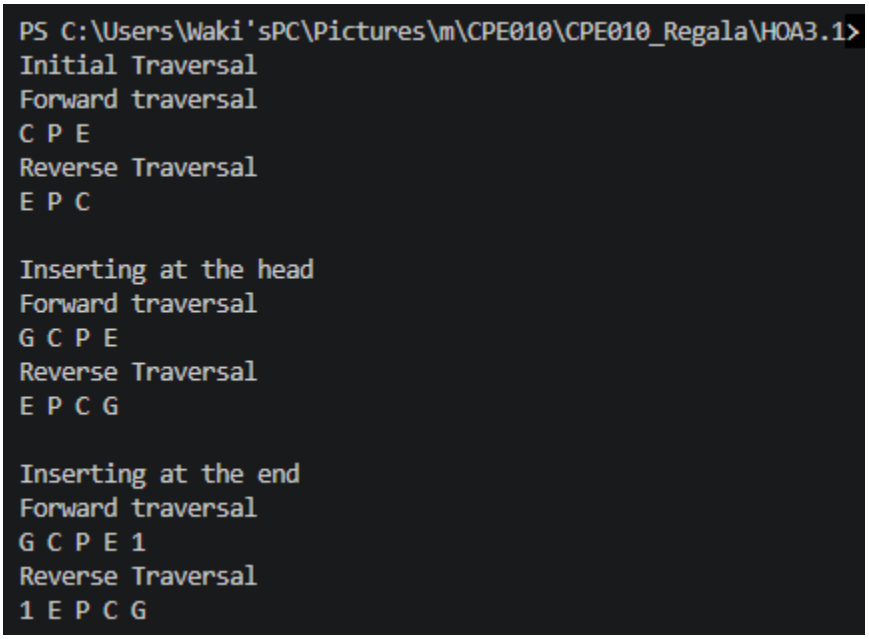
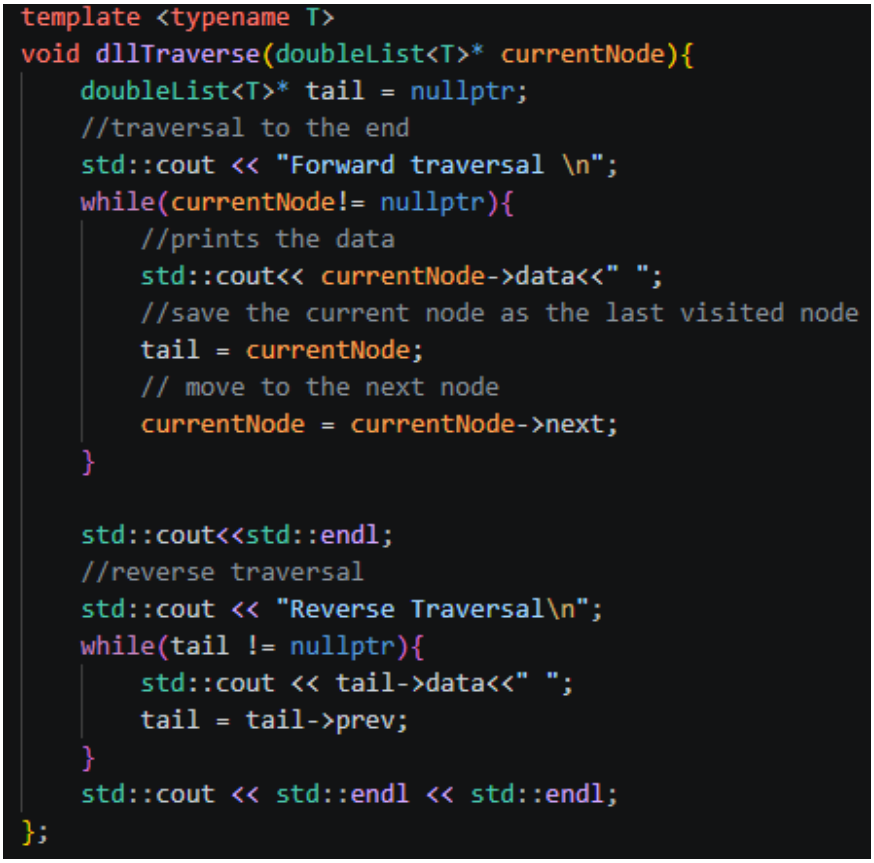
SCREENSHOT	
DISCUSSION	The output shows that the doubly linked list is working correctly by displaying the playlist in both forward and reverse directions. After inserting a new node at the head, the new element becomes the first node, and after inserting at the end, the new element is correctly added as the last node. The reverse traversal also confirms that the prev pointers are properly maintained, demonstrating that the insertion operations preserve the integrity of the doubly linked list.

Table 3-2. Code for the List Operations

Operation	Screenshot
TRAVERSAL	

```
template <typename T>
void dllTraverse(doubleList<T>* currentNode){
    doubleList<T>* tail = nullptr;
    //traversal to the end
    std::cout << "Forward traversal \n";
    while(currentNode!= nullptr){
        //prints the data
        std::cout<< currentNode->data<<" ";
        //save the current node as the last visited node
        tail = currentNode;
        // move to the next node
        currentNode = currentNode->next;
    }

    std::cout<<std::endl;
    //reverse traversal
    std::cout << "Reverse Traversal\n";
    while(tail != nullptr){
        std::cout << tail->data<<" ";
        tail = tail->prev;
    }
    std::cout << std::endl << std::endl;
};
```

INSERTION AT HEAD

```
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {

    // create a new node
    doubleList<T>* newNodePtr = newNode(newData);

    // SET THE new node to point to the current head
    newNodePtr->next = *currentHead;

    // Update previous pointer of old head (verifies)
    if (*currentHead != nullptr){
        (*currentHead)->prev = newNodePtr;
    }

    // Change head pointer to point to new node
    *currentHead = newNodePtr;
}
```

INSERTION AT ANY PART OF THE LIST

```
template <typename T>
void dllInsert(T newData, doubleList<T>* currentNode){

    if(currentNode == nullptr){
        std::cout<<"Previous Node cannoot be null";
        return;
    }

    doubleList<T>* newNode = newNode(newData);

    // connect new node to the next node
    newNode->next = currentNode->next;

    // connect new node to current node
    newNode->prev = currentNode;

    // if not inserting at the end
    if(currentNode->next != nullptr){
        currentNode->next->prev = newNode;
    }

    // connect current node to new node
    currentNode->next = newNode;
}
```

INSERTION AT END

```cpp
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead) {

    // create a new node
    doubleList<T>* newNodePtr = newNode(newData);

    // traverse until the last node
    while (currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    // connect the last node to the new node
    currentHead->next = newNodePtr;

    // connect the prev node to the new node
    newNodePtr->prev = currentHead;
}
```

DELETION OF NODE

```cpp
template <typename T>
void dllDelete(doubleList<T>** currentHead, doubleList<T>* deleteNode){

    // check if list or node is empty
    if(*currentHead == nullptr || deleteNode == nullptr){
        return;
    }

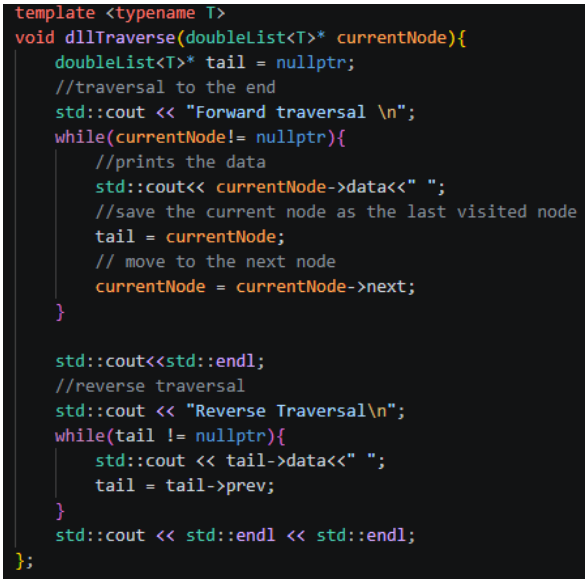
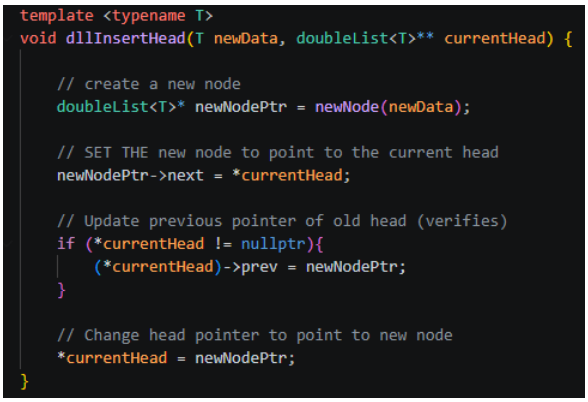
    // if deleting the head
    if(*currentHead == deleteNode){
        *currentHead = deleteNode->next;
    }

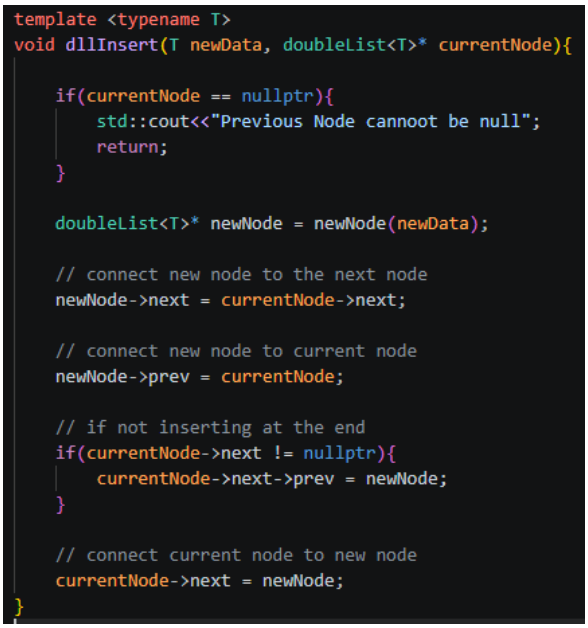
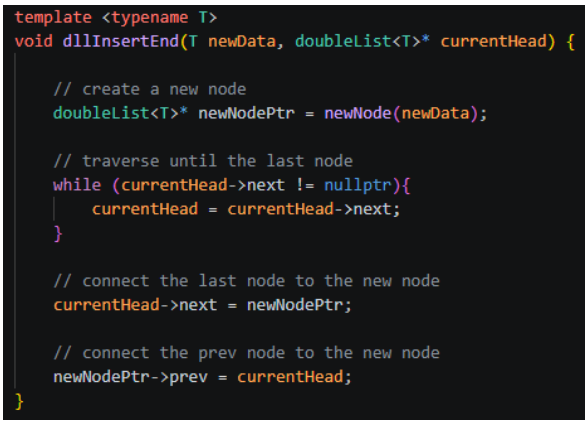
    // update previous node
    if(deleteNode->prev != nullptr){
        deleteNode->prev->next = deleteNode->next;
    }

    if(deleteNode->next != nullptr){
        deleteNode->next->prev = deleteNode->prev;
    }

    delete deleteNode;
}
```

Table 3-4. Modified Operations for Doubly Linked Lists

SCREENSHOT	ANALYSIS
	This function traverses the doubly linked list and displays the nodes in both forward and reverse order. It verifies that the next and prev pointers are correctly linked by printing the list from the head to the tail and then back to the head.
	This function inserts a new node at the beginning of the doubly linked list. It updates the head pointer and adjusts the previous pointer of the old head to maintain the correct links between nodes.

	This function inserts a new node after a specified node in the list. It updates both the next and prev pointers of the surrounding nodes to preserve the structure of the doubly linked list.
	This function adds a new node at the end of the doubly linked list. It first traverses to the last node, then connects the new node while updating the necessary pointers.
	This function removes a specified node from the doubly linked list. It reconnects the neighboring nodes, updates the head if necessary, and frees the memory of the deleted node to maintain the integrity of the list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Main file

```cpp
#include <iostream>
#include <string>
#include "double_LL.h"

int main(){

    doubleList<std::string>* playlist = nullptr;

    // Create Playlist
    insertSong(&playlist, std::string("Sanctuary"));
    insertSong(&playlist, std::string("Get You"));
    insertSong(&playlist, std::string("After Hours"));
    insertSong(&playlist, std::string("Like Him"));

    std::cout << "PLAYLIST\n";
    playPlaylist(playlist);

    // Current Song
    doubleList<std::string>* current = playlist;

    std::cout << "\nCurrent Song: " << current->data << std::endl;

    nextSong(&current);
    std::cout << "Next Song: " << current->data << std::endl;

    nextSong(&current);
    std::cout << "Next Song: " << current->data << std::endl;

    previousSong(&current);
    std::cout << "Previous Song: " << current->data << std::endl;

    // Add Song
    insertSong(&playlist, std::string("Shape of You"));

    std::cout << "\nPlaylist After Adding a Song\n";
    playPlaylist(playlist);

    // Remove Song
    deleteSong(&playlist, std::string("Perfect"));

    std::cout << "\nPlaylist After Removing 'Perfect'\n";
    playPlaylist(playlist);

    return 0;
}
```

DOUBLE.LL.H file

```h
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H
#include <iostream>

template <typename T>
class doubleList{
public:
    T data;
    doubleList<T>* next;
    doubleList<T>* prev;
};

// Create Node
template <typename T>
doubleList<T>* newNode(const T& newData){

    doubleList<T>* temp = new doubleList<T>;

    temp->data = newData;

    // Circular pointers
    temp->next = temp;
    temp->prev = temp;

    return temp;
}


// Insert at End
template <typename T>
void insertSong(doubleList<T>** head, const T& newData){

    doubleList<T>* temp = newNode(newData);

    // Empty playlist
    if(*head == nullptr){
        *head = temp;
        return;
    }

    doubleList<T>* last = (*head)->prev;

    temp->next = *head;
    temp->prev = last;

    last->next = temp;
    (*head)->prev = temp;
}

```

```cpp
// Delete Song
template <typename T>
void deleteSong(doubleList<T>** head, T value){

    if(*head == nullptr)
        return;

    doubleList<T>* current = *head;

    do{

        if(current->data == value){

            // Only one node
            if(current->next == current){

                delete current;
                *head = nullptr;
                return;
            }

            current->prev->next = current->next;
            current->next->prev = current->prev;

            if(current == *head)
                *head = current->next;

            delete current;
            return;
        }

        current = current->next;

    }while(current != *head);

}

```

```
// Play Playlist
template <typename T>
void playPlaylist(doubleList<T>* head){

    if(head == nullptr){

        std::cout << "Playlist is empty.\n";
        return;
    }

    doubleList<T>* current = head;

    std::cout << "\nSongs in Playlist\n";

    do{

        std::cout << current->data << std::endl;
        current = current->next;

    }while(current != head);

}


// Next Song
template <typename T>
void nextSong(doubleList<T>** current){

    if(*current != nullptr)
        *current = (*current)->next;
}


// Previous Song
template <typename T>
void previousSong(doubleList<T>** current){

    if(*current != nullptr)
        *current = (*current)->prev;
}

#endif
```

OUTPUT

```
Songs in Playlist
Sanctuary
Get You
After Hours
Like Him

Current Song: Sanctuary
Next Song: Get You
Next Song: After Hours
Previous Song: Get You

Playlist After Adding a Song

Songs in Playlist
Sanctuary
Get You
After Hours
Like Him
Shape of You

Playlist After Removing 'Perfect'

Songs in Playlist
Sanctuary
Get You
After Hours
Like Him
Shape of You

C:\Users\Waki'sPC\Pictures\m\CPE010\CPE010_Regala\HOA3.1\output>
```

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONe**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.